

Relatório Lista 4: Sistema de autocomple com Trie

Grupo: Angel Jhesua Machado Rodriguez e João Guilherme de Souza Gomes **Professor:** Matheus Werner
Data: Junho de 2026

1 Representação da Trie e definições iniciais

As Tries podem ser retratadas abstratamente de duas maneiras principais. Na primeira, os caracteres das palavras ficam nas arestas e nos nós está o indicador de fim de palavra. Na segunda, os caracteres são guardados diretamente pelos nós, juntamente com os seus outros atributos. A representação da estrutura no código depende de qual conceito é adotado. No presente trabalho, o primeiro modelo foi o escolhido, uma vez que se optou por priorizar a velocidade de execução da busca. Nessa abordagem, o caractere não faz parte explicitamente da estrutura interna do nó, em vez disso, adota-se um alfabeto fixo onde cada nó armazena um array cujas posições são indexadas pelos próprios caracteres do alfabeto.

Definiu-se um “alfabeto” fixo de 36 letras, formado pelas 26 letras do alfabeto latino e os números de 0 a 9. Essa restrição tornou possível que os casos de normalização de chaves ou transformação de caracteres em índices pudessem ser feitos inteiramente através da manipulação da tabela ASCII.

O sistema é *case-insensitive*, para garantir esse comportamento o método `toSearchKey` atua como um normalizador. Antes de qualquer inserção ou busca, as letras da *string* são totalmente convertidas para letras maiúsculas e têm todos os seus caracteres de espaço removidos. Ele se utiliza de outros dois métodos, o primeiro, `removeSpaces`, remove os espaços (através do método `erase` para strings), enquanto o segundo, `toUpperCase`, se aproveita da tabela ASCII que possui a propriedade de os caracteres serem representados por valores inteiros sequenciais, logo, a distância entre qualquer letra minúscula e sua respectiva maiúscula é uma constante equivalente a 32. Ao subtrair o deslocamento calculado por `'a' - 'A'`, o valor binário do caractere é alterado para a sua versão em maiúscula.

O mapeamento de caracteres para os índices do vetor de filhos (`children`) foi implementado no método `charToIndex`. Através da manipulação da tabela ASCII, os dígitos de '0' a '9' são mapeados para as posições de 0 a 9, enquanto as letras de 'A' a 'Z' são deslocadas para ocupar as posições de 10 a 35.

2 Funcionamento dos Métodos `insert`, `contains` e `autocomplete`

A manipulação da árvore Trie ocorre através de três operações principais: inserção de novos elementos, verificação de existência (busca exata) e a recuperação de sugestões baseada em prefixos (autocomplete).

Inserção (`insert`) e Busca Exata (`contains`)

O método de inserção (`insert`) é responsável por preencher a árvore. Ao receber um ponteiro para um objeto `Game`, o algoritmo extrai seu título, o normaliza e inicia uma travessia a partir da raiz (`root`). Para cada caractere da chave, calcula-se o índice correspondente no alfabeto. A travessia ocorre descendo os níveis da árvore; caso o ponteiro no índice correspondente ao caractere atual seja nulo (`nullptr`), um novo `TrieNode` é dinamicamente alocado. Ao atingir o final da *string*, o último nó recebe o ponteiro do objeto `Game` e a flag `isEndOfTitle` é marcada como `true`, construindo o caminho como um título válido no catálogo.

Analogamente, o método `contains` verifica a existência de um jogo específico. Ele percorre a árvore seguindo os índices da chave buscada. Se, em qualquer ponto da descida, o caminho apontar para um nó nulo, o método é interrompido, retornando `false` (indicando que a sequência não existe na Trie). Caso o loop termine e o último nó seja alcançado com sucesso, o método não retorna simplesmente `true`, mas sim o valor da variável `isEndOfTitle` daquele nó. Isso é importante para diferenciar títulos reais de caminhos que são apenas prefixos de palavras maiores (por exemplo, garantir que buscar por “HALF” retorne `false` se apenas “HALF LIFE” estiver cadastrado).

Busca por Prefixo (`autocomplete`)

A operação de autocomplete utiliza recursão para encontrar os nós desejado. Pode ser dividida em três partes principais. A primeira, utiliza o método auxiliar `takeNode` para caminhar da raiz até o nó que representa

o final do prefixo digitado pelo usuário. Se o prefixo não for encontrado na árvore, o método retorna um ponteiro nulo e o processo é interrompido, retornando um vetor vazio. Uma vez localizado o nó do último caractere do prefixo, na segunda parte chamamos a função de busca em profundidade, um método recursivo chamado `recursiveDFS`. A cada chamada da função, caso o nó não seja nulo, verifica-se a *flag isEndOfTitle*, se tiver valor `true` o ponteiro do jogo correspondente é inserido no vetor de resultados `games`. Em seguida, um loop percorre o vetor de ponteiros filhos do nó atual e uma nova chamada recursiva para cada filho válido é feita, selecionando todos os jogos da subárvore do prefixo. A terceira e última parte se trata da ordenação, feita pelo método `sortResults` seguindo os critérios antes mencionados, e da seleção dos `k` primeiros dados no vetor de resultado, feita com um loop simples.

3 Análise de Custo Computacional

Denotemos por L , o tamanho do título do jogo após a conversão para chave de busca; p , o tamanho do prefixo buscado após a conversão; s , o número de nós visitados na subárvore delimitada pelo prefixo; e r , o número total de jogos encontrados dentro dessa mesma subárvore.

- **Inserção (insert) - $\mathcal{O}(L)$:** O custo computacional da inserção é linear em relação ao tamanho da chave de busca. O algoritmo executa um laço que itera exatamente L vezes, processando um caractere por iteração. Em cada passo, a conversão do caractere para índice, a verificação da existência do nó e a eventual alocação de memória ocorrem em tempo constante $\mathcal{O}(1)$ devido ao uso do *array* de filhos estático. Logo, o custo total depende somente de L .
- **Busca Exata (contains) - $\mathcal{O}(L)$:** Análogamente, a busca exata depende unicamente do comprimento da palavra procurada. No pior caso (quando a palavra existe ou quando a divergência ocorre apenas no último caractere), o algoritmo desce L níveis na árvore. Como o acesso a cada transição de caractere é feito através de indexação direta em $\mathcal{O}(1)$, a complexidade final é dada por $\mathcal{O}(L)$.
- **Busca por Prefixo (autocomplete) - $\mathcal{O}(p + s + r \log r)$:** A complexidade pode ser descrita usando a seguinte divisão:
 1. **Navegação inicial:** O método `takeNode` consome $\mathcal{O}(p)$ operações para descer até o nó do prefixo.
 2. **Exploração recursiva:** A função `recursiveDFS` visita exatamente s nós na subárvore. Como a verificação de filhos em cada nó é feita sobre um tamanho fixo (36), o custo de varredura dessa estrutura é linear em relação ao número de nós visitados, ou seja, $\mathcal{O}(s)$.
 3. **Ordenação e extração:** A ordenação dos r jogos encontrados consome $\mathcal{O}(r \log r)$ através do *Merge Sort*, enquanto o laço de extração final executa em tempo linear $\mathcal{O}(k)$, sendo assintoticamente absorvido pelo custo da ordenação.

4 Comparação e Uso de Memória

Trie versus Solução Ingênua

Uma solução ingênua para o problema de *autocomplete* consistiria em armazenar todos os jogos em um grande vetor (*array*) linear. A cada busca realizada, o sistema precisaria percorrer a lista completa de jogos, checando individualmente se o título de cada um deles começa com o prefixo digitado. Se o catálogo possuir N jogos cadastrados, o custo de pior caso para essa varredura seria de aproximadamente $\mathcal{O}(N \times p)$. Embora essa estratégia funcione para bases de dados muito pequena, ela se torna inviável à medida que o sistema escala para milhares ou milhões de registros.

Com a árvore Trie, a grande vantagem é a eliminação do fator N na etapa de localização do prefixo. O tempo necessário para alcançar a subárvore correspondente depende unicamente do comprimento do próprio prefixo procurado (p).

O Custo de Memória Estrutural

Em contrapartida, a Trie exige um consumo de memória consideravelmente superior ao de um vetor simples. O tamanho do alfabeto escolhido influencia no dimensionamento desse custo de memória. Como a

estrutura foi projetada com um tamanho fixo (`ALPHABET_SIZE = 36`), cada nó individual da árvore instancia obrigatoriamente um vetor interno com 36 ponteiros.

Considerando uma arquitetura de 64 bits, onde cada ponteiro consome 8 *bytes*, cada nó da nossa Trie ocupa pelo menos $36 \times 8 = 288$ *bytes* de memória. Esse espaço é alocado integralmente no momento da criação do nó, mesmo que a maioria dessas posições permaneça vazia (`nullptr`). Caso tivéssemos optado por um alfabeto estendido (como a tabela ASCII padrão de 256 caracteres), o tamanho de cada nó saltaria para mais de 2 *kilobytes*.

Redução de Nós por Compartilhamento de Prefixos

Apesar do custo individual de cada nó ser elevado, a Trie se consolida como uma estrutura viável e inteligente graças ao mecanismo de **compartilhamento de prefixos**.

Em um catálogo de jogos, é natural que muitos títulos comecem com as mesmas palavras ou sequências de caracteres. Títulos como “Halo”, “Hades” e “Half-Life”, por exemplo, reaproveitam exatamente os mesmos nós iniciais correspondentes às letras “H” e “A”. A árvore não duplica caminhos idênticos; ela apenas ramifica quando os títulos divergem. Esse reaproveitamento geométrico reduz o número total de nós necessários na árvore à medida que o volume de dados inserido cresce e se torna mais denso, compensando o desperdício dos ponteiros nulos e equilibrando a relação entre espaço e desempenho.